

1. Priority Queue

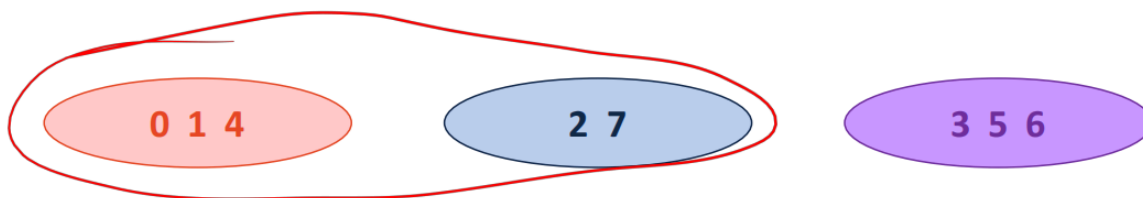
Method	insert	removeMin	build
unsorted array/list	$O(1)$	$O(n)$	$O(n)$
sorted array/list	$O(n)$	$O(1)$	$O(n \log n)$
AVL Tree	$O(\log n)$	$O(\log n)$	$O(n \log n)$
Heap	$O(\log n)$	$O(\log n)$	$O(n)$

2. Disjoint Set

2.1 Implementation

2.1.1 Implementation #1

Implementation #1



AI识图

代表元

0	1	2	3	4	5	6	7
0	0	2	3	0	3	3	2

0 0

Find(k): $O(1)$

Union(k1, k2): $O(n)$

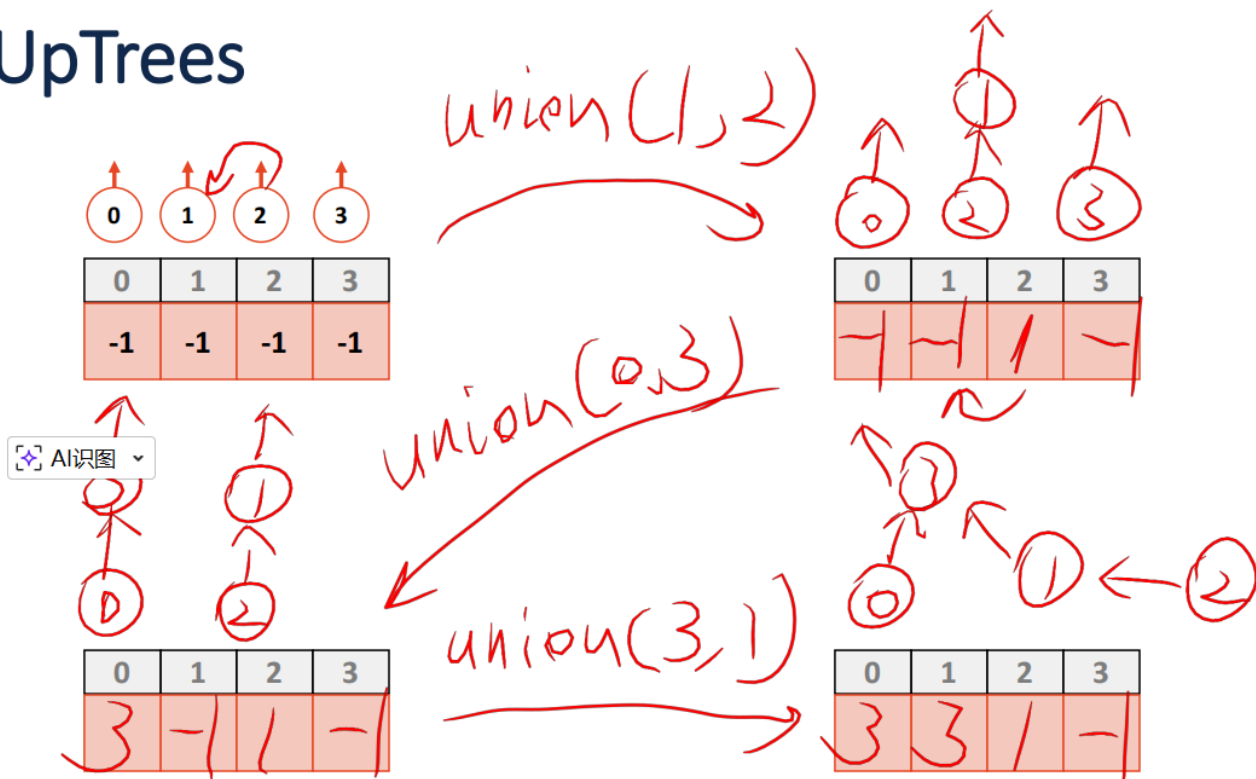
2.1.2 Implementation #2: UpTrees

Definition of an array:

- The index represents an element
- If $\text{arr}[i] < 0$, then i is a leaf node (representing an element)
- If $\text{arr}[i] \geq 0$, then $\text{arr}[i]$ stores the index of i 's parent node

Forming the uptree:

UpTrees



Finding in uptree:

```
int DisjointSets::find(int i) {  
    if (s[i] < 0) { return i; }  
    else { return find(s[i]); }  
}
```

2.2 Union

2.2.1 Basic Union

```
void DisjointSets::union(int r1, int r2){  
    s[r2]=r1;  
}
```

2.2.2 Smart Union

1. By height
 - Hang the shorter tree beneath the taller one
2. By size
 - Nest a smaller set within a larger set

Both guarantee the height of the tree is: $O(\log n)$

Note that:

Common UpTree Encoding Methods:

- The root node stores a negative number
- Non-root nodes store the parent's index

If using union by size :

- The absolute value of the negative number at the root indicates the size of the set

For example, if the root value is -6 :

- This indicates that the set has 6 elements

code:

```

void DisjointSets::unionBySize(int root1, int root2) {
    int newSize = arr_[root1] + arr_[root2];

    if (arr_[root1] < arr_[root2]) {
        arr_[root2] = root1;
        arr_[root1] = newSize;
    } else {
        arr_[root1] = root2;
        arr_[root2] = newSize;
    }
}

```

2.3 Path Compression

find doesn't just return once it finds the root;
it also connects the nodes along the path directly to the root.

2.3.1 code

```

int find(int i) {
    if (arr_[i] < 0) return i;
    arr_[i] = find(arr_[i]);
    return arr_[i];
}

```

2.3.2 Analysis

Any sequence of m union and find operations results in a worst-case running time of $O(m \log^* n)$

The prerequisites here are:

- Use of smart union
- Use of path compression in find